Python: exercises on loops with strings and lists

This notebook was generated in **solutions** mode.

Exercise 1: Shortest Words

Exercise Description

Shortest Words

Write a function `shortest_words` that:

- takes a list of words
- returns a list containing the shortest words in the list received as argument. The list will containt more than one word when there are multiple words with the same length.

Examples:

- `shortest_words([])` returns `[]`
- `shortest_words(['sheldon', 'cooper'])` returns `['cooper']`
- `shortest_words(['sheldon', 'cooper', 'howard'])` returns `['cooper', 'howard']`

NOTE: do not use any buit-in function from Python to solve the exercise

Your solution

```
def shortest_words(lista_stringhe):
    lunghezza_minima = float('inf')
    for parola in lista_stringhe:
        lunghezza_parola = len(parola)
        if lunghezza_parola < lunghezza_minima:
            lunghezza_minima = lunghezza_parola

    parole_corte = []
    for parola in lista_stringhe:
        if len(parola) == lunghezza_minima:
            parole_corte.append(parola)

    return parole_corte
```

Test your solution

```
assert shortest_words([]) == []
assert sorted(shortest_words(['sheldon', 'cooper'])) == ['cooper']
assert sorted(shortest_words(['sheldon', 'cooper', 'howard'])) == ['cooper', 'howard'
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: find how many equal numbers

Exercise Description

find how many equal numbers

Define a function `count_equals` that:

- takes as arguments four numbers
- returns:
    - the maximum number of equal numbers between the four

Example:

- `count_equals(1,2,3,4)` should return `0`
- `count_equals(1,2,5,4)` should return `0`
- `count_equals(1,2,2,2)` should return `3` because there are three `2` in the sequence
- `count_equals(1,1,1,2)` should return `3` because there are three `1` in the sequence

Your solution

```
def count_equals(a, b, c, d):
    # Create a list 'numbers' containing the four input values
    numbers = [a, b, c, d]

    # Initialize 'max_count' to store the highest occurrence of any number
    max_count = 0

    # Iterate over each 'num' in the 'numbers' list
    for num in numbers:
        # Initialize a 'count' variable to track how many times 'num' appears
        count = 0

        # Iterate over each 'other' number in 'numbers' to compare with 'num'
        for other in numbers:
            # If 'num' is equal to 'other', increment 'count' by 1
```

```
            if num == other:
                count += 1

        # If the current 'count' is greater than 'max_count', update 'max_count'
        if count > max_count:
            max_count = count

    # Return 'max_count' if it's greater than 1 (indicating duplicates exist)
    # Otherwise, return 0 (no duplicates found)
    return max_count if max_count > 1 else 0
```

Test your solution

```
assert count_equals(1,2,3,4) == 0
assert count_equals(1,5,3,4) == 0
assert count_equals(1,5,5,4) == 2
assert count_equals(1,5,1,4) == 2
assert count_equals(1,5,5,5) == 3
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Fibonacci's sequence

Exercise Description

Fibonacci's sequence

Define a function `fibonacci` that:

- takes as arguments:
    - an integer number `n`
- returns:
    - a list containing the first `n` numbers of the Fibonacci's sequence

NOTE: The Fibonacci sequence is a series of numbers where each number is the sum of the two previous ones, starting with 0 and 1. To calculate it, you begin with 0 and 1, then add these to get the next number. Continue this process to generate the sequence. It goes 0, 1, 1, 2, 3, 5, 8, and so on.

Your solution

```
def fibonacci(n):
    # Initialize a list 'fib_sequence' with the first two Fibonacci numbers: 0 and 1
    fib_sequence = [0, 1]
```

```
    # If 'n' is 1, return a list containing only the first Fibonacci number [0]
    if n == 1:
        return [0]

    # Loop from 2 to 'n-1' to generate the next Fibonacci numbers
    for i in range(2, n):
        # Calculate the next Fibonacci number as the sum of the last two numbers in t
        next_fib = fib_sequence[-1] + fib_sequence[-2]
        # Append the new Fibonacci number to the 'fib_sequence' list
        fib_sequence.append(next_fib)

    # Return the first 'n' numbers of the Fibonacci sequence (in case 'n' is less tha
    return fib_sequence[:n]
```

Test your solution

```
assert fibonacci(1) == [0]
assert fibonacci(3) == [0,1,1]
assert fibonacci(7) == [0, 1, 1, 2, 3, 5, 8]
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Collatz

Exercise Description

Collatz

Define a function `collatz` that:

- takes as argument an integer number `n`
- returns:
    - a list containing all the numbers generated by the Collatz conjecture (stopping when reaching `1`)

NOTE: The Collatz Conjecture is a mathematical problem that starts with any positive integer. The process involves two steps: if the number is even, divide it by 2; if it's odd, multiply it by 3 and add 1. Repeat this process with the resulting number. The conjecture suggests that, no matter what number you start with, you'll eventually reach the number 1.

Your solution

```
def collatz(n):
    # Initialize an empty list 'sequence' to store the numbers in the Collatz sequenc
```

```
    sequence = []

    # Continue looping until 'n' becomes 1
    while n != 1:
        # If 'n' is even, divide it by 2
        if n % 2 == 0:
            n = n // 2
        # If 'n' is odd, update 'n' to 3*n + 1
        else:
            n = 3 * n + 1
        # Append the new value of 'n' to the sequence
        sequence.append(n)

    # Return the generated Collatz sequence
    return sequence
```

Test your solution

```
assert collatz(12) == [6,3,10,5,16,8,4,2,1]
assert collatz(1) == []
assert collatz(2) == [1]
assert collatz(4) == [2,1]
assert collatz(19) == [58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16,
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Find missing number in a sequence

Exercise Description

Find missing number in a sequence

Define a function `find_missing` that:

- Takes a list of `n-1` integers, which represents a sequence of numbers from 1 to `n`, but one number is missing.
- Returns the missing number.

NOTE: Suppose that there is always only one number missing

Example:

- `find_missing([1, 2, 4, 5])` should return `3`.
- `find_missing([2, 3, 4, 6, 1])` should return `5`.

Your solution

```python
def find_missing(nums):
    # Calculate the expected length of the complete list, which includes one missing
    n = len(nums) + 1

    # Calculate the total sum of the first 'n' natural numbers using the formula n(n
    total_sum = n * (n + 1) // 2

    # Calculate the actual sum of the numbers present in the input list 'nums'
    actual_sum = sum(nums)

    # The missing number is the difference between the total sum and the actual sum
    return total_sum - actual_sum
```

Test your solution

```python
assert find_missing([1, 2, 4, 5]) == 3
assert find_missing([2, 3, 4, 6, 1]) == 5
```

Test your solution

```python
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Prefixes of a String

Exercise Description

1. Define the string `s` equal to `The Big Bang Theory`
2. Create the empty list `p`
3. Using a loop, add all prefixes of `s` to `p`
4. Print `p`

Your solution

```
s = "The Big Bang Theory"
p = []
for i in range(len(s)):
    p.append(s[:i+1])
print("The list of prefixes is", p)
```

Test your solution

```
assert len(p) == 19
```

Test your solution

```
assert p[0] == "T"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Check Palindrome

Exercise Description

Check Palindrome

Define a function `is_palindrome` that:

- Takes a string as input.
- Returns `True` if the string is a palindrome, and `False` otherwise.

NOTE: A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward (**ignoring spaces, punctuation, and capitalization**).

Example:

- `is_palindrome("racecar")` should return `True`.
- `is_palindrome("hello")` should return `False`.
- `is_palindrome("A man, a plan, a canal, Panama")` should return `True`.

Your solution

```
def is_palindrome(s):
    # Initialize an empty string 's2' to hold the filtered and normalized characters
    s2 = ""

    # Iterate over each character in the input string 's'
    for i in range(len(s)):
        # Check if the character is an alphabetic character
        if s[i].isalpha():
            # Convert the character to lowercase and add it to 's2'
            s2 += s[i].lower()

    # Check if the filtered string is a palindrome by comparing characters
    for i in range(0, int(len(s2) / 2)):
        # If characters at symmetric positions do not match, it's not a palindrome
        if s2[i] != s2[len(s2) - i - 1]:
            return False

    # If all symmetric characters match, return True indicating it's a palindrome
    return True
```

Test your solution

```
assert is_palindrome("racecar")
assert not is_palindrome("hello")
assert is_palindrome("A man, a plan, a canal, Panama")
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Check Postfixes of a String

Exercise Description

1. Define the string `s` equal to `The Big Bang Theory`
2. Define the list `p` equal to `["y", "ry", "ery", ""]`
3. Remove from `p` any string that is not a postfix of `s`
4. Print `p`

Your solution

```
s = "The Big Bang Theory"
p = ["y", "ry", "ery", ""]
to_be_removed = []
for postfix in p:
    if not s.endswith(postfix):
        to_be_removed.append(postfix)
for postfix in to_be_removed:
    p.remove(postfix)
print("The list of postfixes is", p)
```

Test your solution

```
assert p == ["y", "ry", ""]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Prime Numbers

Exercise Description

Define a function `is_prime` that:

- takes as arguments a list `L` of positive integers
- returns:
    - if the list is not empty: a new list where the i-th element is a boolean asserting whether the i-th element from `L` is a prime number
    - otherwise: `[]`

Your solution

```
def is_prime_number(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

def is_prime(L):
    L2 = []
    for x in L:
        L2.append(is_prime_number(x))
    return L2
```

Test your solution

```
assert is_prime([]) == []
```

Test your solution

```
assert is_prime([3, 4, 9, 11]) == [True, False, False, True]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Word Frequency

Exercise Description

Define a function `count_freq` that:

- takes as arguments:
    - a string `s`
    - a list `L` of words
- returns:
    - if the list is not empty: a new list where the i-th element is the number of occurences in `s` of the i-th word from `L`
    - otherwise: `[]`

Your solution

```
def count_freq(s, L):
    counts = []
    for x in L:
        counts.append(s.count(x))
    return counts
```

Test your solution

```
assert count_freq("test", []) == []
```

Test your solution

```
assert count_freq("Aejeje", ["e", "je", "aje"]) == [3, 2, 0]
```

Debug your solution